End-to-End Data Management Pipeline for a Movie Recommendation System

BITS ID: 2025AG05520

1. Objective and design

This project implements the Data Management for Machine Learning Assignment 2 using the MovieLens Latest Small dataset. The goal is a repeatable workflow that validates and prepares interaction data, creates reusable features, trains two recommendation approaches, and records comparable offline metrics.

GitHub repository: recommendation-pipeline: https://github.com/Tango9100/recommendation-pipeline

The implementation uses modular Python scripts and a fail-fast root orchestrator. Project paths and model/evaluation settings are centralized in config/config.yml. CSV is used for transparent intermediate artifacts, and MLflow uses a local SQLite tracking store.

2. Data and processing

The input consists of 9,742 movie records and 100,836 user ratings from 610 users. MovieLens provides user identifiers but no demographic profile fields; therefore the user dimension table records IDs only and user activity features are derived from training interactions.

The validation stage checks required columns, missing values, duplicate rows, numeric/integer field types, rating bounds (0.5–5.0), convertible timestamps, and rating-to-movie references. Invalid rows are saved in data/errors/; the current dataset produced zero invalid type, rating, timestamp, or reference records and no missing or duplicate values.

Preparation standardizes identifiers and ratings, removes invalid records, and separates each user’s latest interaction into test data. This yields 100,226 training interactions and 610 held-out interactions. Transformations include user activity summaries, item popularity, and a user-item matrix.

3. Feature management and models

Feature engineering creates user features (average rating, activity count, unique movies, recency), item features (popularity, unique users, average rating, title and genres), and interaction features (rating, timestamp, hour, day of week, and month). Features used for training are generated from the training split to avoid leakage from held-out interactions. Their definitions are listed in reports/feature_catalog.csv.

The baseline popularity recommender ranks movies by training rating count and average rating, excluding items with fewer than five training ratings. The personalized recommender uses item-based collaborative filtering with cosine similarity and user rating history. Both models persist artifacts under models/.

4. Evaluation and results

The models are compared at K=5 on all 610 users with held-out interactions. Metrics are Precision@5, Recall@5, NDCG@5, catalog Coverage, and genre-based Diversity.

Model | Precision@5 | Recall@5 | NDCG@5 | Coverage | Diversity

Popularity | 0.0052 | 0.0262 | 0.0145 | 0.0077 | 0.7790

Collaborative Filtering | 0.0062 | 0.0311 | 0.0186 | 0.0612 | 0.7373

Collaborative filtering is higher on precision, recall, NDCG, and coverage for this run; popularity has higher diversity. The absolute ranking metrics are low, so the results should be treated as a reproducible baseline rather than evidence of production-quality recommendations. These recommenders rank items and do not predict ratings, so RMSE and MAE are not applicable.

5. Orchestration, tracking, and limitations

Run python run_pipeline.py from the repository root to execute ingestion, validation, preparation, transformation, feature engineering, both training stages, evaluation, and MLflow tracking in dependency order. A failure stops subsequent stages. MLflow records dataset and split parameters, K, ranking metrics, and model/report artifacts in the configured SQLite store.

The main limitations are the absence of user demographic attributes, reliance on one public dataset, and weak top-five accuracy. The models are offline recommenders, not a deployed service; feature storage is CSV-based rather than a production feature store. A future iteration should evaluate alternative splits and ranking algorithms and add temporal/user preference features from an appropriate data source.